

Structure Demotion: Reclaiming the KV-Cache Budget from Repeated Separators under Extreme Compression

Satyajeeth Suresh Kannan
Innovation Academy, Alpharetta, Georgia, USA
jeetusk08@gmail.com

Abstract

Query-agnostic KV-cache eviction methods such as KVzip score every cached token by how much it helps the model reconstruct its own context. At extreme compression this criterion misallocates the budget: repeated structural tokens—newlines, list markers, separators—are easy to reconstruct from one another and keep a large share of the cache, while repeated content words, exactly the evidence that aggregation tasks need, are discounted. We propose *structure demotion*, a training-free rule that multiplies the scores of structural tokens (tokens containing no letter or digit) that occur at least four times in the context by 0.25 before eviction, while exempting “joiners” inside identifiers such as the dashes of a UUID. Applied to RestoreKV+, the strongest entry on the kvpress leaderboard, at compression ratio 0.9375 (one sixteenth of the cache kept) on RULER-4096 with Qwen3-8B, structure demotion raises the mean score from 86.38 to 89.64 (+3.25 points; paired 95% confidence interval [+2.73, +3.77]), recovering 36% of the gap to the uncompressed model. Seven of the thirteen tasks improve significantly and none degrades significantly; common-words extraction gains 12.1 points. We also report an instructive failure: demoting digits, or the dashes inside UUIDs, destroys needle retrieval. The rule is a default-off option of about thirty lines in the kvpress library.

1 Introduction

The key-value (KV) cache of a transformer grows linearly with context length and dominates inference memory for long inputs. KV-cache eviction keeps only a subset of the cached keys and values. Query-agnostic methods compress the cache once, before any question is known, so that the compressed cache can be reused across queries. KVzip [1] is the reference method in this setting: it asks the model to reconstruct its context and keeps the KV pairs that receive the most attention during that reconstruction. Its successors refine the scoring [2, 3] or add learned restore tokens that recover what eviction loses [4]. On the public kvpress leaderboard [5, 6], the strongest method for Qwen3-8B at compression ratios 0.88 and 0.9375 is RestoreKV+ [4].

At compression ratio 0.9375, where only one of every sixteen cached positions survives, RestoreKV+ still loses 8.9 of the 95.3 points that Qwen3-8B [7] scores on RULER-4096 [8] with a full cache. The loss is concentrated: common-words and frequent-words extraction and the multi-needle tasks account for most of it. We traced it to what the reconstruction criterion rewards. A token that the model can easily reconstruct from its neighbours is not necessarily unimportant, and a token the model keeps because other copies predict it is not necessarily useful. In long structured contexts the cheapest tokens to keep “in the loop” of reconstruction are the repeated separators: newlines, list numbers, dashes and quotes. They crowd out the content words the question needs, and repeated content words—which reconstruction discounts because later copies are predictable—are exactly the answer in aggregation tasks.

We make three contributions.

- A diagnosis of where the budget goes under reconstruction scoring at extreme compression (Section 3).
- *Structure demotion*, a one-line change to the eviction scores with a single exemption rule, implemented as a default-off option of `KVzipPress` and `RestoreKVPress` in `kvpress` (Section 4).
- A full-benchmark evaluation with paired confidence intervals against the published leaderboard predictions, plus the ablations and the failure case that shaped the rule (Section 5).

2 Background

Compression ratio. Following `kvpress`, the compression ratio is the fraction of cached context positions that are removed; at 0.9375 the model keeps 6.25% of the context KV pairs, averaged over layers and heads.

KVzip. After prefilling the context, `KVzip` [1] runs the model on a prompt that asks it to repeat the context and records, for every cached KV pair, the maximum attention it receives from the reconstruction queries. Pairs are then evicted by a global bottom- k over all layers and heads, so the budget is non-uniform across layers. The `KVzip+` normalization introduced with `KVzap` [2] rescales these attention weights by the norm of the value vector after the output projection and by the inverse norm of the query-side hidden state.

RestoreKV. `RestoreKV` [4] appends $n = 8$ learned restore tokens that attend to the full cache in one LoRA-adapted pass before eviction; the resulting restore cache occupies part of the same budget, and `KVzip`-style pruning fills the rest. `RestoreKV+` is `RestoreKV` with `KVzip+` normalization. The restore embeddings and LoRA weights are trained by self-distillation from the full-cache model and are published for Qwen3-8B.

RULER-4096. `RULER` [8] contains thirteen synthetic long-context tasks: eight needle-in-a-haystack variants (single, multi-key, multi-value, multi-query), variable tracking (`vt`), common-words and frequent-words extraction (`cwe`, `fwe`) and two question-answering tasks (`qa_1`, `qa_2`). The `kvpress` benchmark uses 500 examples per task at a 4,096-token context and scores each task by string matching (the fraction of reference answers found in the output; any reference for QA). The leaderboard reports the unweighted mean over tasks.

3 Where the budget goes

We instrumented the eviction step of `RestoreKV+` on a common-words extraction example (5,127 tokens; the model must list the ten most frequent words in a numbered list of words). Table 1 shows how the 6.04% of the cache that survives is spent. List-index numbers make up 60% of the context and take 38% of the budget; whitespace and newline tokens make up 16% of the context and are kept at almost the same rate as words (8.7% versus 10.0%), taking 23% of the budget. Only 38% of the budget goes to words.

The reconstruction criterion also discounts repetition. Averaged over the words of this example, the first three occurrences of a word score about 0.045 and are kept at a rate of 12.7%; from the fourth occurrence on, the score halves (about 0.022–0.026) and the kept rate drops to 5.3–6.5%.

token class	share of context	RestoreKV+		+ structure demotion	
		kept	share of budget	kept	share of budget
words	23.0%	10.0%	38%	13.0%	50%
numbers (list indices)	60.4%	3.8%	38%	4.3%	43%
whitespace / newlines	16.2%	8.7%	23%	2.4%	6%
other punctuation	0.4%	10.7%	1%	13.6%	1%
gold-word occurrences kept			6.9%		9.7%
score on this example			90		100

Table 1: Budget allocation on one common-words extraction example (Qwen3-8B, compression ratio 0.9375, 6.04% of the context kept in both cases). Structure demotion moves budget from newlines to words and raises the retention of the answer words by 41%.

A fourth copy of a word is predictable from the first three, so reconstruction needs it less. On a counting task this is backwards: each of the ten gold words occurs 30 times, and their mean score (0.026) is below the mean score of all word tokens (0.037). Only 6.9% of the gold-word occurrences survive.

4 Structure demotion

Let $x_1, \dots, x_n$ be the token ids of the context and $s_{\ell,h,i}$ the eviction score of position i in layer ℓ and KV head h . We call a token id *structural* if its decoded string, with surrounding whitespace removed, is not a word, word piece or number, that is, it does not fully match the pattern $[\^W_](?:[\^W_] | ['\ -])^*$ (a letter or digit followed by letters, digits, apostrophes or hyphens). Newlines, list punctuation, quotes, brackets and lone dashes are structural; words, word pieces, digits and numbers are not. Position i is *demoted* when

1. x_i is structural;
2. the id x_i occurs at least m times in the context (we use $m = 4$, the occurrence at which the reconstruction score halves in Section 3); and
3. x_i is not a *joiner*: a structural token without whitespace whose previous token ends with a letter or digit and whose next token starts with one, such as the dashes of a UUID, the point of 3.14, the colon of 10:30 or the comma of 1,000.

Before the global bottom- k eviction we set $s_{\ell,h,i} \leftarrow \alpha s_{\ell,h,i}$ for every demoted position, in every layer and head, with $\alpha = 0.25$. Positions before the end of the chat prefix and the attention-sink positions are never demoted, and RestoreKV’s restore tokens lie outside the context and are untouched. The number of evicted pairs is unchanged, so the achieved compression ratio is identical (6.04% kept in the example of Table 1 with and without demotion).

The rule needs only the token ids and the tokenizer that KVzip already loads. It is a single pass over the context on the CPU, far cheaper than KVzip’s reconstruction passes. With $\alpha = 0$ the option is disabled and every existing press is bit-identical.

Why joiners are exempt. The first full-benchmark run used conditions (1) and (2) only. It scored 64.6 on `niah_multikey_3`, against 97.0 for the baseline. In that task the keys and values are UUIDs, which the Qwen tokenizer splits into letter pieces, digits and standalone dash tokens; the

task	RestoreKV+	+ demotion	Δ	95% CI	better / worse
cwe	78.88	90.96	+12.08	[+11.08, +13.14]	362 / 12
fwe	83.67	90.00	+6.33	[+5.07, +7.67]	104 / 9
niah_multikey_1	85.80	92.20	+6.40	[+3.40, +9.40]	45 / 13
niah_multikey_2	99.60	99.20	-0.40	[-1.20, +0.40]	1 / 3
niah_multikey_3	97.00	96.80	-0.20	[-1.40, +1.00]	4 / 5
niah_multiquery	96.80	98.20	+1.40	[+0.55, +2.25]	43 / 14
niah_multivalue	82.75	88.15	+5.40	[+3.70, +7.15]	148 / 55
niah_single_1	100.00	100.00	0.00	[0.00, 0.00]	0 / 0
niah_single_2	90.80	96.00	+5.20	[+2.80, +7.60]	32 / 6
niah_single_3	93.00	95.80	+2.80	[+0.40, +5.20]	27 / 13
qa_1	65.40	67.60	+2.20	[-0.60, +5.00]	32 / 21
qa_2	49.20	50.40	+1.20	[-2.00, +4.00]	33 / 28
vt	100.00	100.00	0.00	[0.00, 0.00]	0 / 0
mean	86.38	89.64	+3.25	[+2.73, +3.77]	

Table 2: RULER-4096, Qwen3-8B, compression ratio 0.9375. RestoreKV+ scores are the published leaderboard entry; confidence intervals and example counts come from the paired per-example comparison (the Δ for qa_2 is +1.00 against the recomputed baseline of 49.40). For reference, the uncompressed model scores 95.32 and RestoreKV without KVzip+ normalization 81.93 at the same ratio.

dash repeats hundreds of times per context, so it was demoted and the UUIDs were broken up by eviction. Exempting joiners reduces the number of demoted positions in a typical `niah_multikey_3` context from 566 to 172 and restores the task to 96.8, while leaving the demoted set unchanged on the extraction and variable-tracking tasks. Separators that carry whitespace on at least one side (list markers, newlines, quotes) are still demoted.

5 Experiments

Setup. We evaluate Qwen3-8B in bfloat16 on the kvpress RULER-4096 benchmark (13 tasks $\times$ 500 examples) at compression ratio 0.9375, using the published RestoreKV+ adapters for Qwen3-8B and the kvpress evaluation script unchanged except for the new option (`RestoreKVPress(kvzip_plus_normalization=True, structure_demotion=0.25)`). The run took about 35 hours on an Apple M5 Pro laptop (PyTorch MPS backend, SDPA attention). Leaderboard entries were produced on CUDA with FlashAttention; on the first 100 common-words examples our local run of the unmodified baseline scored 77.2, in line with the published 78.88 over all 500.

Paired comparison. Because the leaderboard publishes per-example predictions, we compare against RestoreKV+ on exactly the same 6,500 examples. We recompute per-example scores with the leaderboard’s metric (the recomputation reproduces the published task scores exactly for twelve tasks and within 0.2 points for qa_2) and bootstrap the per-example differences within each task (10,000 resamples) to obtain 95% confidence intervals for each task and for the 13-task mean.

Development protocol. All design choices—the structural test, m , α and the joiner exemption—were made on paired 100-example screens (the first 100 examples of a task) and, for the joiner exemption, after the failure of the first full run described in Section 4. The final configuration was then run once on the full benchmark.

variant (100-example screens)	cwe	fwe	mk_1	mv	vt	qa_1	qa_2
RestoreKV+ (local baseline)	77.2	82.3	90.0	80.8	100.0	67.0	60.0
demote structural tokens, $\alpha = 0.25$ (ours)	90.6	90.0	94.0	85.5	100.0	64.0	56.0
demote structural tokens, $\alpha = 0.5$	87.4	–	–	–	–	–	57.0
also demote repeated digits	96.1	90.0	0.0	0.0	–	–	–
demote repeated non-alphabetic bigrams (≥ 16)	90.0	–	90.0	–	–	–	–
boost repeated content words instead	58.0	–	–	–	–	–	–

Table 3: Development screens on the first 100 examples of each task (Qwen3-8B, ratio 0.9375; mk_1 = niah_multikey_1, mv = niah_multivalue; – = not run). The joiner exemption, added later, leaves the demoted set unchanged on cwe, fwe and vt and changes at most 12 positions per context on the others. With 100 examples a single task score has a standard error of several points; the QA losses seen here did not reappear on the full benchmark (Table 2).

Main result. Table 2 shows the full comparison. Structure demotion raises the mean from 86.38 to 89.64, a gain of 3.25 points with a paired 95% confidence interval of $[+2.73, +3.77]$ (no bootstrap resample reached zero). It closes 36% of the gap between RestoreKV+ and the uncompressed model (95.32). Seven tasks improve significantly; the largest gains are on the two extraction tasks (cwe +12.1, fwe +6.3) and on the needle tasks whose haystacks are lists or repeated sentences (niah_multikey_1, niah_single_2, niah_multivalue, each above +5). The two UUID tasks move by -0.4 and -0.2 , both well inside their intervals, and no task is significantly worse. The QA tasks, whose contexts are prose with little repeated structure, change little.

Ablations. Table 3 summarises the development screens. Halving the demotion ($\alpha = 0.5$) gives back 3.2 points on cwe for 1 point on qa_2, so we kept $\alpha = 0.25$. Demoting repeated digits as well recovers cwe almost to the uncompressed level (96.1) but scores zero on the needle tasks: the Qwen tokenizer splits every number into single digits, so a seven-digit needle is seven tokens that each “repeat”, and losing one of them makes the answer wrong. A rule based on repeated non-alphabetic bigrams (for example "2" ".") protected needles but gained less on cwe. The opposite intervention—undoing the discount on repeated content words—made things worse (-19 on cwe): the repeated tokens it boosted were mostly the list-index digits, which moved budget in the wrong direction. The lesson is that any repetition-based rule must be checked on needle tasks, whose answers are made of digits and identifier pieces.

6 Related work

Attention-based eviction. H2O keeps “heavy-hitter” tokens with large accumulated attention [9]; StreamingLLM keeps attention sinks and a recent window [10]; SnapKV selects positions using the attention of an observation window at the end of the prompt [11]; PyramidKV allocates smaller budgets to higher layers [12]; TOVA keeps the tokens with the highest current attention [13]. These methods score tokens with the queries of the prompt itself, which ties the compressed cache to one question.

Query-agnostic compression. KVzip replaces the question with a context-reconstruction task [1], Fast KVzip approximates it with a lightweight gate [3], KVzap approximates KVzip+ scores with a fast surrogate model [2], and RestoreKV adds learned restore tokens [4]. Structure

demotion is orthogonal to all of them: it post-processes whatever reconstruction scores they produce.

Separators. SepLLM observes that separator tokens attract disproportionate attention and trains models that compress each segment into its separator [14]. Our finding is complementary: in an off-the-shelf model scored by reconstruction, *repeated* separators are retained far beyond their value for the downstream task, and moving their share of an extreme budget to content tokens helps.

7 Limitations

We evaluated one model, one context length and one compression ratio at full scale, on one benchmark whose contexts are synthetic and highly structured; the gains are largest exactly where contexts repeat separators, and we expect little effect on plain prose. The run used Apple’s MPS backend with SDPA attention rather than the CUDA and FlashAttention setup of the other leaderboard entries; the baseline reproduced within two points locally, and a CUDA re-run needs only the released option. The factor α and the threshold m were set on small screens and not tuned further. The structural test is a tokenizer-level heuristic: in code, JSON or languages where punctuation carries meaning, demoting repeated symbols may remove information the task needs, and tokenizers that merge punctuation with letters would see fewer structural tokens. Finally, the joiner exemption was added after observing the failure of the first full run, so the full-benchmark number evaluates a rule that was revised once in response to that benchmark.

8 Conclusion

Reconstruction scoring asks which tokens the model needs in order to reproduce its context; at extreme compression the answer is dominated by repeated scaffolding. Demoting repeated structural tokens by a constant factor, while protecting punctuation inside identifiers, reallocates that budget to content and improves the strongest query-agnostic method on RULER-4096 by 3.25 points at one-sixteenth of the cache. The rule is training-free, model-free and a few lines long, which makes it easy to combine with future reconstruction-based presses.

Code and data

The implementation is the `structure_demotion` option of `KVzipPress/RestoreKVPress`, submitted to `kvpress` as pull request `NVIDIA/kvpress#288` (branch `structure-demotion-restorekv` of <https://github.com/JeetuSK0808/kvpress>). Predictions, metrics and configuration for the full run are in the leaderboard submission <https://huggingface.co/spaces/nvidia/kvpress-leaderboard/discussions/24>.

Use of AI assistance

The experiments, analysis and code were developed with an AI coding assistant (Claude Code, Anthropic) directed by the author, who reviewed the results and the manuscript.

Acknowledgments

We thank the kvpress maintainers for the benchmark, the leaderboard and its published predictions, and the RestoreKV authors for releasing their adapters.

References

- [1] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction. arXiv preprint arXiv:2505.23416, 2025. <https://arxiv.org/abs/2505.23416>
- [2] Simon Jegou and Maximilian Jeblick. KVzap: Fast, Adaptive, and Faithful KV Cache Pruning. arXiv preprint arXiv:2601.07891, 2026. <https://arxiv.org/abs/2601.07891>
- [3] Jang-Hyun Kim, Dongyoon Han, and Sangdoo Yun. Fast KVzip: Efficient and Accurate LLM Inference with Gated KV Eviction. arXiv preprint arXiv:2601.17668, 2026. <https://arxiv.org/abs/2601.17668>
- [4] Changwoo Baek, Seungjun Shin, and Kyeongbo Kong. RestoreKV: Recovering Full-Cache Behavior Under Aggressive Query-Agnostic KV Cache Eviction. arXiv preprint arXiv:2608.01247, 2026. <https://arxiv.org/abs/2608.01247>
- [5] NVIDIA. kvpress: LLM KV cache compression made easy. <https://github.com/NVIDIA/kvpress>, 2026. Software, accessed September 2026.
- [6] NVIDIA. kvpress leaderboard. <https://huggingface.co/spaces/nvidia/kvpress-leaderboard>, 2026. Accessed September 2026.
- [7] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [8] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the Real Context Size of Your Long-Context Language Models?. arXiv preprint arXiv:2404.06654, 2024. <https://arxiv.org/abs/2404.06654>
- [9] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, et al. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv preprint arXiv:2306.14048, 2023. <https://arxiv.org/abs/2306.14048>
- [10] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. arXiv preprint arXiv:2309.17453, 2023. <https://arxiv.org/abs/2309.17453>
- [11] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, et al. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv preprint arXiv:2404.14469, 2024. <https://arxiv.org/abs/2404.14469>

- [12] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, et al. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv preprint arXiv:2406.02069, 2024. <https://arxiv.org/abs/2406.02069>
- [13] Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are Multi-State RNNs. arXiv preprint arXiv:2401.06104, 2024. <https://arxiv.org/abs/2401.06104>
- [14] Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, et al. SepLLM: Accelerate Large Language Models by Compressing One Segment into One Separator. arXiv preprint arXiv:2412.12094, 2024. <https://arxiv.org/abs/2412.12094>